

神奇的动态规划 – 培训手册

🌟 神奇的动态规划 🌟

—— 写给刚小学毕业的你 · 算法启蒙第一课

📖 本手册配套：课件（HTML 幻灯片）+ 代码（Python）👩👨 适合人群：刚小学毕业、对编程和数学感兴趣的同学 ⌚ 建议学习时间：每天 1 小时，大约 5~7 天读完

📖 目录

- 第 0 章：先听个小故事
- 第 1 章：从爬楼梯认识动态规划
- 第 2 章：动态规划的”三步法”和”三要素”
- 第 3 章：第一类模板 —— 一维动规
 - 3.1 最大子段和
 - 3.2 最长上升子序列 (LIS)
- 第 4 章：第二类模板 —— 二维表格动规
 - 4.1 最长公共子序列 (LCS)
 - 4.2 0/1 背包
 - 4.3 完全背包
- 第 5 章：第三类模板 —— 区间动规
 - 5.1 石子合并游戏
- 第 6 章：总结与挑战
- 📄 附录：动规速查卡片

第 0 章：先听个小故事

小明要去春游。他有一个小背包，**最多能装 10 斤**。商店里有 4 种好吃的：

物品	价值	重量
🍪 饼干	1	2 斤
🍏 苹果	3	3 斤
🍫 巧克力	9	7 斤
🧀 奶酪	5	4 斤

他想让背包里的东西**总价值最高**，该怎么挑？

如果瞎试，得把所有挑法都列一遍：4 种东西，每种”拿”或”不拿”，一共 $2 \times 2 \times 2 \times 2 = 16$ 种挑法。这还只是 4 种东西，要是 30 种东西呢？那就是 $2^{30} \approx 10$ 亿种，电脑都得算半天！😱

有没有更聪明的办法？**有！** 这就是这本书要教你的 **动态规划**（英文叫 Dynamic Programming，简称 DP）。

💡 **一句话理解动态规划：** 别傻傻地一遍遍重复算，把算过的答案”记下来”，下次直接用！ 就像你背了九九乘法表，以后算 7×8 就不用再一个一个加了。

准备好了吗？我们出发！🚀

第 1 章：从爬楼梯认识动态规划

🏠 问题：爬楼梯

有一段楼梯，你一次能上 1 阶，或者上 2 阶。爬到第 n 阶，一共有多少种不同的爬法？

比如爬到第 1 阶：只能”上 1 阶”，**1 种**。爬到第 2 阶：可以”1+1”，也可以”直接上 2 阶”，**2 种**。

那爬到第 3 阶呢？第 4 阶呢？

🧩 秘诀：从最小的开始，找规律

动态规划有一个**超级重要**的思路：

💡 **把大问题，拆成小问题。先解决小的，再用小的答案推出大的。**

我们来想：你要爬到第 n 阶，你最后一步是从哪里跨上来的？只有两种可能： - 从第 $n-1$ 阶跨 1 阶上来 🐾 - 从第 $n-2$ 阶跨 2 阶上来 🐾

所以！

爬到第 n 阶的走法数 = 爬到第 $n-1$ 阶的走法数 + 爬到第 $n-2$ 阶的走法数

我们一格一格填：

楼梯阶数	1	2	3	4	5	6
走法数	1	2	3	5	8	13

- 第 3 阶 = 第 2 阶 + 第 1 阶 = $2 + 1 = 3$ ✓
- 第 4 阶 = 第 3 阶 + 第 2 阶 = $3 + 2 = 5$ ✓
- 第 5 阶 = $5 + 3 = 8$ ✓

你发现了吗？这就是大名鼎鼎的斐波那契数列（兔子数列）！🐰

🔑 关键思想（记住这句话）：把复杂问题拆成小问题；把小问题的答案存起来，避免重复计算，再用来求大问题的答案。——这就是动态规划！

第 2 章：动态规划的”三步法”和”三要素”

📋 三步法（每次做动规题，照着做）

步骤	做什么	爬楼梯的例子
①	写状态转移方程（找规律）	第 n 阶 = 第 $n-1$ 阶 + 第 $n-2$ 阶
②	给最初的状态做初始化	第 1 阶 = 1，第 2 阶 = 2
③	按方程一步步算出结果	第 3 阶 = $2+1=3$ ，第 4 阶 = $3+2=5$

🔥 名词解释（别怕，就是名字而已）： - 状态：每一步算出来的”答案”。比如”第 n 阶的走法数”就是一个状态。 - 状态转移方程：从一个状态推到下一个状态的”公式”。 - 状态转移：从上一状态算到下一状态的过程。

🧱 三要素（一个问题能不能用动规，看这三点）

1. **最优子结构** 🏗️ 大问题的最优解，能由小问题的最优解拼出来。（比如：要知道”上 10 阶”的最优解，得先知道”上 9 阶”和”上 8 阶”的。）
2. **无后效性** 🗑️ 推当前问题时，只用到前面小问题的答案，不用管它是怎么来的。（比如：算第 5 阶时，我只要知道第 4 阶 = 5，不用管那 5 种走法具体长啥样。）
3. **重叠子问题** 🔄 同一个小问题会被反复用到。把它的答案存起来，就能省下大量重复计算。（比如：算第 10 阶要用到第 8 阶，算第 9 阶也要用到第 8 阶 —— 第 8 阶被用了两次！）

✅ **什么时候用动规？** 当问题有**最优子结构**和**无后效性**时，就能用动规！如果还有**重叠子问题**，那存答案省时间的效果就特别明显。

第 3 章：第一类模板 —— 一维动规

这类问题的答案，只用**一排格子**（一维数组 `dp[]`）就能存下。

3.1 最大子段和（连续选数字，让和最大）

🎯 问题

给你一串数字，例如：

1 -2 5 3 -4 5

你要从中挑出一段连续的数字，让它们的和最大。这个最大的和，就叫**最大子段和**。

比如：- 选 1 -2 5，和 = 4 - 选 5 3 -4 5，和 = 9 🤩（看起来不错！）- 选 5 3，和 = 8

答案到底是多少呢？

💡 思路

设 `dp[i]` = 以第 i 个数字”结尾”的最大子段和。

想清楚一件事：第 i 个数字，要么**接着前面那段**，要么**自己单独开一段**。- 接着前面：`dp[i-1] + a[i]` - 自己开一段：`a[i]`

哪个大，就选哪个！

状态转移方程：

```
dp[i] = max( dp[i-1] + a[i] , a[i] )
```

初始： `dp[0] = a[0]`

算一算（数字序列 1, -2, 5, 3, -4, 5）

i	a[i]	dp[i] = max(dp[i-1]+a[i], a[i])	说明
0	1	1	第一个数，自己开段，= 1
1	-2	-1	$\max(1-2, -2) = \max(-1, -2) = -1$
2	5	5	$\max(-1+5, 5) = \max(4, 5) = 5$ （自己开段更划算）
3	3	8	$\max(5+3, 3) = 8$ （接着前面！）
4	-4	4	$\max(8-4, -4) = 4$
5	5	9	$\max(4+5, 5) = 9$

最后答案是所有 `dp[i]` 里最大的那个 = 9 🎉 对应连续段是 5 3 -4 5（和正好是 9）。

代码：见 `代码/02_最大子段和.py`

3.2 最长上升子序列（LIS）

问题

给你一串数字，例如：

5 3 9 11 4 2 15

你可以从中挑一些数（不用挨着，但前后顺序不能乱），让它们从左到右越来越大。挑出来的最长能有多长？这就是最长上升子序列（Longest Increasing Subsequence, LIS）。

例子： - 5 3 9 11 —— 不是上升的（5 到 3 变小了）❌ - 5 9 11 15 —— 一直在变大！长度 4

✅ - 3 9 11 15 —— 也行！长度 4 ✅

💡 思路

设 $dp[i]$ = 以第 i 个数字”结尾”的最长上升子序列长度。

关键：看看第 i 个数前面，有哪些数比它小，那些数后面都能接上 $a[i]$ ，长度 +1。在所有能接的里面挑最长的，再 +1。

🔍 状态转移方程：

```
对所有  $j < i$  且  $a[j] < a[i]$ :  
     $dp[i] = \max(dp[i], dp[j] + 1)$   
初始：每个  $dp[i] = 1$  (至少能选自己这一个数)
```

🧮 算一算 (5, 3, 9, 11, 4, 2, 15)

位置 i	1	2	3	4	5	6	7
数字 $a[i]$	5	3	9	11	4	2	15
长度 $dp[i]$	1	1	2	3	2	1	4

- $a[3]=9$ ：前面比它小的有 5、3（长度都是 1），所以 $dp[3] = 1+1 = 2$
- $a[4]=11$ ：前面比它小且最长的 dp 是 $dp[3]=2$ ，所以 $dp[4] = 2+1 = 3$
- $a[7]=15$ ：前面最长的是 $dp[4]=3$ ，所以 $dp[7] = 3+1 = 4$ 🎉

答案是最大的 dp 值 = 4。

📄 代码：见 [代码/03_最长上升子序列.py](#)

🔔 敲黑板：最大子段和 vs 最长上升子序列 || 最大子段和 | 最长上升子序列 || -|-|-| | 选出来的数字 | 必须连续 | 可以不连续 || $dp[i]$ 含义 | 以 i 结尾的最大和 | 以 i 结尾的最大长度 |

第 4 章：第二类模板 —— 二维表格动规

这类问题光靠一排格子不够，得画一张表格（二维数组 $dp[i][j]$ ）来存答案。

4.1 最长公共子序列 (LCS)

🎯 问题

给你两个序列，找它们都拥有的最长子序列，越长越好。（注意：子序列不用连续，但前后顺序不能乱。）

例子：

```
s1 = a b c b d a b
s2 = b d c a b a
```

它们的最长公共子序列是 `b c b a` 或 `b d a b`，长度都是 4。

💡 思路：画一张大表格

设 $dp[i][j]$ = s1 的前 i 个字符 和 s2 的前 j 个字符 的最长公共子序列长度。

填表时，看 s1 的第 i 个字符和 s2 的第 j 个字符一不一样：

情况 ①：两个字符相同 ✅

```
dp[i][j] = dp[i-1][j-1] + 1
```

（就像两个相同的积木对上了，把这一对拿掉，前面的最长长度 +1）

情况 ②：两个字符不同 ❌

```
dp[i][j] = max( dp[i-1][j] , dp[i][j-1] )
```

（这一对对不上，那就分别试试“丢掉 s1 的这个”或“丢掉 s2 的这个”，取较大的）

📊 表格 (s1 = abcbdad, s2 = bdcaba)

表格第一行第一列全是 0（空序列和谁比都是 0）。

dp[i][j]	↓ s2 \ s1 →	a	b	c	b	d	a	b
	(空)	0	0	0	0	0	0	0
b		0	1	1	1	1	1	1
d		0	1	1	1	2	2	2
c		0	1	2	2	2	2	2
a		1	1	2	2	2	3	3
b		1	2	2	3	3	3	4
a		1	2	2	3	3	4	4

右下角的 4 就是答案 🎉

代码：见 [代码/04_最长公共子序列.py](#)

4.2 0/1 背包（去春游，背包装什么最值？）

🎯 问题

回到开头小明春游的故事：

物品	🎨 价值	📏 重量
① 饼干	1	2
② 苹果	3	3
③ 巧克力	9	7
④ 奶酪	5	4

背包最多装 10 斤。每种物品只有 1 件，要么拿（1）要么不拿（0）—— 所以叫 0/1 背包。让总价值最高，该怎么拿？

💡 思路：还是画表格

设 $dp[i][j]$ = 前 i 种物品、背包载重为 j 时能装的最大价值。

对第 i 种物品，拿还是不拿：

- 不拿：价值 = 上一种物品在同样载重下的最大价值 = $dp[i-1][j]$
- 拿：要先给它腾出重量，剩下 $j - w[i]$ 的载重，再加它的价值 = $dp[i-1][j-w[i]] + v[i]$ （注意：拿它，载重得够，也就是 $j \geq w[i]$ ）

状态转移方程：

当 $j < w[i]$ 时 (装不下): $dp[i][j] = dp[i-1][j]$
当 $j \geq w[i]$ 时 (装得下): $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$

表格 (载重 0~10)

物品 (重量, 价值)	0	1	2	3	4	5	6	7	8	9	10
① (2,1)	0	0	1	1	1	1	1	1	1	1	1
② (3,3)	0	0	1	3	3	4	4	4	4	4	4
③ (7,9)	0	0	1	3	3	4	4	9	9	10	10
④ (4,5)	0	0	1	3	5	5	6	9	9	10	12

右下角 12 就是答案 🎉 拿 ②苹果 + ③巧克力 (重量 $3+7=10$, 价值 $3+9=12$)。

🔍 小插曲：为什么不用“贪心”？

“贪心法”就是：每次都挑看起来最划算的。比如先挑价值最高的巧克力 (价值 9)。巧的是，这道题贪心碰巧也得到 12。

但是！贪心不保证永远对。换个数字，贪心可能就翻车了 (第 4.3 节你就会看到一个例子)。而动态规划，永远给你正确答案。这就是它的厉害之处！💪

📄 代码：见 `代码/05_01背包.py` (还附赠了“一维数组省空间”的优化版本)

4.3 完全背包 (物品想拿多少拿多少)

🎯 问题

还是那 4 种物品、载重 10 的背包。但这次规则变了：每种物品都有无限件，想拿几件拿几件 (0 件、1 件、2 件……都行)。这叫 完全背包。

	0/1 背包	完全背包
每种物品数量	只有 1 件	无限件
“拿”的时候，看哪一行	看上一行 <code>dp[i-1][...]</code>	看本行 <code>dp[i][...]</code>

为什么“拿”的时候要看本行？因为拿完一件之后，还能再拿同一件！所以是“在本行已经拿过的基础上，再加一件”。

当 $j \geq w[i]$ 时: $dp[i][j] = \max(dp[i-1][j], dp[i][j-w[i]] + v[i])$
 ↑不拿 ↑本行再加一件

注意第二个是 `dp[i][j-w[i]]`，不是 `dp[i-1][j-w[i]]`！这是和 0/1 背包唯一的区别！

物品 (重量, 价值)	0	1	2	3	4	5	6	7	8	9	10
① (2,1)	0	0	1	1	2	2	3	3	4	4	5
② (3,3)	0	0	1	3	3	4	6	6	7	9	9
③ (7,9)	0	0	1	3	3	4	6	9	9	10	12
④ (4,5)	0	0	1	3	5	5	6	9	10	10	12

答案也是 12（这道题恰好和 0/1 背包答案一样）。

比如贪心挑“单位价值最高”的： – 巧克力 $9 \div 7 \approx 1.29$ （单位价值最高） – 苹果 $3 \div 3 = 1$

贪心会先拿巧克力，再拿苹果，得到 12。看起来还行？

但是！如果把巧克力的价值改成 10，贪心还是先拿巧克力（10）+苹果（3）= 13。可实际上，拿 3 个苹果（ $3 \times 3 = 9$ 斤，价值 9）.....嗯这里也是 13 更大。真正翻车的例子更复杂，关键结论是：

⚠ 完全背包不能用贪心法保证最优解，必须用动态规划！

🧙 一维数组的小魔法

完全背包用一维数组时，内层循环要从前往后（正序）遍历载重；而 0/1 背包用一维数组时，要从后往前（逆序）遍历。这个小细节决定了“能不能重复拿同一件物品”。（代码里有详细注释哦）

📄 代码：见 `代码/06_完全背包.py`

第 5 章：第三类模板 —— 区间动规

这类问题是关于“一段连续的区间”的，思路是：先合并小区间，再拼成大区间。

5.1 石子合并游戏

🎯 问题

地上有 4 堆石子排成一排，石子数分别是：

[1]	[2]	[3]	[4]
5	3	3	5

规则：每次只能选相邻的两堆合并成一堆，得分 = 合并后这堆的石子数。一直合并到只剩 1 堆。问：最小总得分是多少？

比如先合并 [1][2] ($5+3=8$ ，得 8 分)，变成 [8, 3, 5]一直合并下去。

💡 思路：把大区间拆成小区间

设 $dp[i][j]$ = 把第 i 堆到第 j 堆合并成一堆的最小得分。

怎么合并 $[i..j]$ 这一段？最后一步一定是：把它分成左右两半，分别合并好，再合到一起。而最后合并的代价 = 这一段所有石子的总和（不管怎么分，最后大合并的代价都一样）。

所以：枚举“在哪里切一刀”（切点 k ），把区间分成 $[i..k]$ 和 $[k+1..j]$ ：

状态转移方程：

```
dp[i][j] = min( dp[i][k] + dp[k+1][j] + 第i到j堆的总和 )  
(k 从 i 枚举到 j-1, 取最小的)
```

一步步填表

第 0 步：长度 1 的区间（只有一堆，不用合并，得分 0） 对角线全填 0。

第 1 步：长度 2 的区间（两堆直接合并，得分 = 两堆之和） - $dp[1][2] = 5+3 = 8$ - $dp[2][3] = 3+3 = 6$ - $dp[3][4] = 3+5 = 8$

第 2 步：长度 3 的区间 - $dp[1][3]$ （第 1~3 堆，和 = $5+3+3 = 11$ ）： - 切在 1 后面： $dp[1][1] + dp[2][3] + 11 = 0 + 6 + 11 = 17$ - 切在 2 后面： $dp[1][2] + dp[3][3] + 11 = 8 + 0 + 11 = 19$ - 取最小 = 17 ✓
- $dp[2][4]$ （第 2~4 堆，和 = $3+3+5 = 11$ ）： - 切在 2 后面： $dp[2][2] + dp[3][4] + 11 = 0 + 8 + 11 = 19$ - 切在 3 后面： $dp[2][3] + dp[4][4] + 11 = 6 + 0 + 11 = 17$ - 取最小 = 17 ✓

第 3 步：长度 4 的区间（整个！和 = $5+3+3+5 = 16$ ） - $dp[1][4]$ ： - 切在 1 后面： $dp[1][1] + dp[2][4] + 16 = 0 + 17 + 16 = 33$ - 切在 2 后面： $dp[1][2] + dp[3][4] + 16 = 8 + 8 + 16 = 32$ ✨ - 切在 3 后面： $dp[1][3] + dp[4][4] + 16 = 17 + 0 + 16 = 33$ - 取最小 = 32 🎉

最终表格：

$dp[i][j]$	j=1	j=2	j=3	j=4
i=1	0	8	17	32
i=2		0	6	17
i=3			0	8
i=4				0

答案：最小总得分 = 32 🎉

代码：见 [代码/07_石子合并.py](#)

🧠 区间动规的精髓：按“区间长度”从小到大算，先算所有长度 2 的，再算长度 3 的.....最后算整个长度。每个大区间，都由它“切一刀”分成的两个小区间拼出来。

第 6 章：总结与挑战

🏆 你已经学会了 7 个经典动规问题！

#	问题	核心方程	答案（课上的例子）
1	爬楼梯	$f(n)=f(n-1)+f(n-2)$	第6阶=13
2	最大子段和	$dp[i]=\max(dp[i-1]+a[i], a[i])$	9
3	最长上升子序列	$dp[i]=\max(dp[j]+1), a[j]<a[i]$	4
4	最长公共子序列	相同+1；不同取 $\max(\text{上}, \text{左})$	4
5	0/1 背包	$\max(\text{不拿}, \text{拿}=dp[i-1][j-w]+v)$	12
6	完全背包	$\max(\text{不拿}, \text{拿}=dp[i][j-w]+v)$	12
7	石子合并	$\min(dp[i][k]+dp[k+1][j]+\text{区间和})$	32

🎯 动态规划”三步法”（终极版）

- ① 找状态转移方程（找规律！这是灵魂）
- ② 初始化（最开头的几个状态先定好）
- ③ 按方程一步步算（小的算完算大的）

🎮 挑战题（试试看！）

1. 爬楼梯进阶：如果一次能上 1 阶、2 阶 或 3 阶，到第 5 阶有多少种走法？（提示：第 n 阶 = 第 $n-1$ + 第 $n-2$ + 第 $n-3$ 阶）
2. 最大子段和变式：序列 $-2, 1, -3, 4, -1, 2, 1, -5, 4$ 的最大子段和是多少？（答案：6，选 $4, -1, 2, 1$ ）
3. 背包变式：物品价值 $[3, 4, 5, 6]$ 、重量 $[2, 3, 4, 5]$ 、载重 5 的 0/1 背包，最大价值是多少？（答案：7，选第 1、2 个：价值 $3+4=7$ ，重量 $2+3=5$ 正好装满 ✅）
4. 石子合并变式：石子堆 $[1, 3, 5, 2]$ ，最小合并得分是多少？（答案：22，自己画表验证一下～）

💡 做不出来没关系！把每个小问题的答案记下来、一步一步推，这就是动态规划的全部秘密。

🌟 最后的话

动态规划看起来难，其实就一句话：

把大问题拆成小问题，把小问题的答案记下来，不重复劳动。

这不仅是写代码的技巧，更是一种聪明的思考方式。生活里很多事都能这么拆解：- 写一篇大作文 📝 → 拆成“开头、中间、结尾”一小段一小段写 - 整理乱糟糟的房间 🧹 → 一个区域一个区域收拾 - 背一首长诗 📖 → 一句一句背，背过的不再重背

恭喜你读完了这本手册！👏 接下来打开课件跟着老师再过一遍，再跑一跑代码亲手试试，你就是小小算法达人啦！🚀💡

附录：动规速查卡片

动态规划 (DP) 三步法速查卡

- ① 写状态转移方程（找规律）
- ② 初始化最前面的状态
- ③ 按方程从小到大一步步算

三要素：最优子结构 / 无后效性 / 重叠子问题

一维：最大子段和、最长上升子序列

二维：最长公共子序列、0/1背包、完全背包

区间：石子合并

0/1 背包一维：j 逆序 完全背包一维：j 正序

—— 完 ——